# Lenguajes de Programación

## Tercera prueba presencial

## Fecha: 13 de junio de 2022

**Ejercicio 1 (5 puntos).** Contemplamos la creación de un documentador de código. El código entrante estará compuesto por un conjunto de prototipos de funciones en lenguaje c cuyos parámetros y salida serán siempre de los tipos primitivos *int* o *float* (o *void* en el caso de las salidas si la función no devuelve nada). El código resultante será el mismo que el de entrada, pero incluyendo comentarios de bloque con la siguiente información para cada función, en el formato especificado en el ejemplo:

a) Nombre de función
b) Parámetros y su tipo
c) Tipo del resultado

| Ejemplo de código de entrada | Salida |
|---|---|
| int suma_valores(int a, int b);<br>void muestra_menu();<br>float operar(float x); | /*<br>Nombre función: suma_valores<br>Parámetros de entrada:<br>a: int<br>b: int<br>Tipo del resultado: int<br>*/<br>int suma_valores(int a, int b);<br><br>/*<br>Nombre función: muestra_menu<br>Parámetros de entrada: ninguno<br>Tipo del resultado: sin resultado<br>*/<br>void muestra_menu();<br><br>/*<br>Nombre función: operar<br>Parámetros de entrada:<br>x: float<br>Tipo del resultado: float<br>*/<br>float operar(float x); |

**Se pide producir las especificaciones jflex y/o java cup necesarias para construir el documentador. No es necesario contemplar situaciones de error. Puedes asumir que el código de entrada es siempre correcto.**

Con respecto a jflex, se pide la especificación completa. Se recuerdan a continuación algunos de los elementos sintácticos que podrían ser relevantes:

```
%line
%column
%char
%standalone o %cup
%{          %}
%init{      %init}
%eof{       %eof}
%state
%xstate
```

En referencia a la especificación java cup, **no** es necesario incluir la sección correspondiente al bloque parser code con el método main que instancia el analizador sintáctico. Deberás considerar las siguientes secciones:

| | |
|---|---|
| - Clases adicionales en el bloque action code | action code {:<br>:} |
| - Declaración de símbolos terminales y no terminales. | terminal …;<br>non terminal …; |
| - Declaración de precedencias, en el caso de ser necesarias. | precedence left …; |
| - Gramática con sus acciones correspondientes, en la sintaxis específica de java cup. | … ::= …:... {: :} |

**Se consideran requisitos fundamentales e indispensables para la evaluación de la solución los siguientes:**

- **El cumplimiento estricto de la especificación.**
- **La presencia de comentarios y explicaciones que ayuden a la comprensión del código. Todas las variables que utilices deberán aparecer apropiadamente comentadas.**
- **La declaración de símbolos terminales y no terminales en java cup, con sus tipos correspondientes.**
- **La consistencia de los tipos de todos los símbolos (terminales y no terminales) con las asignaciones a RESULT.**

**No se aceptarán (se evaluarán como 0), aquellas soluciones que no cumplan estrictamente con <u>todos</u> estos aspectos.**

**Ejercicio 2 (1 punto).** Sin definir otras funciones adicionales, implementar una función en lisp llamada *convertir_a_pares* que, dadas dos listas del mismo tamaño suministradas como parámetros, devuelva una nueva lista de tuplas, cada una compuesta por los elementos de cada lista que ocupan la misma posición en las listas suministradas como parámetros.

(convertir_a_pares '(1 2 3 4) '(5 6 7 8)) → ((1 5) (2 6) (3 7) (4 8))

**Ejercicio 3 (2 puntos).** Sin definir otras funciones adicionales, implementar una función en lisp llamada *aplicar_f_listas* que, dada una función y un número arbitrario de listas sin anidamientos suministradas como parámetro, devuelva una nueva lista compuesta de tantas listas como hayan sido suministradas como parámetros, en las que la función haya sido aplicada sobre todos los elementos de todas las listas.

(aplicar_f_listas (lambda(x) (* 2 x)) '(1 2 3) '(2 3 4)) → ((2 4 6) (4 6 8))

(aplicar_f_listas #'1+ '(1 2 3) '(2 3 4) '(1 1 1)) → ((2 3 4) (3 4 5) (2 2 2))

**Ejercicio 3 (2 puntos).** Sin definir otras funciones adicionales, implementar una función en lisp llamada filtrar_lista que, dado un elemento y una lista de listas, elimine de cada lista todos los elementos menores que el elemento proporcionado.

(filtrar_lista 3 '((1 2 3) (2 3 4 5) (5 4 3 2 1))) → ((3) (3 4 5) (5 4 3))

(filtrar_lista 0 '((1 2 3) (2 1) (8 9))) → ((1 2 3) (2 1) (8 9))

(filtrar_lista 10 '((1 2 3) (2 1) (8 9))) → (NIL NIL NIL)

**Recordatorio funciones LISP**

**Aritméticas**

(+ $s_1$ $s_2$ $s_3$ ... $s_n$) (- $s_1$ $s_2$ $s_3$ ... $s_n$)
(* $s_1$ $s_2$ $s_3$ ... $s_n$) (/ $s_1$ $s_2$ $s_3$ ... $s_n$)
(1+ s) (1- s)
(MAX $s_1$ $s_2$ $s_3$ ... $s_n$) (MIN $s_1$ $s_2$ $s_3$ ... $s_n$)
(FLOOR $s_1$) (CEIL $s_1$)
(MOD $s_1$ $s_2$)

**Listas**

(LIST $e_1$ $e_2$ … $e_n$) Devuelve una lista formada por los elementos especificados
(CAR lista) Retorna el primer elemento
(CDR lista) Retorna la cola de la lista
(CONS elemento lista) Devuelve nueva lista cuyo primer elemento es elemento y cuya cola es lista
(APPEND $lista_1$ $lista_2$ … $lista_n$) Concatenación de listas
(REVERSE lista) Invertir los elementos
(LENGTH lista) Devuelve el número de elementos
(REMOVE e l) Elimina las apariciones del elemento e de la lista l

**Comprobación de tipos**

(ATOM arg) – ¿Es un átomo? T y NIL lo son
(LISTP arg) – ¿Es una lista?
(NUMBERP arg) – ¿Es un número?

**Comprobación de valor**

(ZEROP arg) – Devuelve T si arg es 0
(MINUSP arg) – Devuelve T si arg es negativo
(NULL arg) – Devuelve T si arg es NIL

**Comparación de valores**

(EQUAL $arg_1$ $arg_2$)
(MEMBER Elemento Lista)
(< $arg_1$ $arg_2$ $arg_3$ … $arg_n$)
(>$arg_1$ $arg_2$ $arg_3$ … $arg_n$)
(=$arg_1$ $arg_2$ $arg_3$ … $arg_n$)

**Predicados lógicos**

(NOT s)
(AND $s_1$ $s_2$ $s_3$ ... $s_n$)
En el caso de evaluar todos a verdadero, devuelve el último valor obtenido (el de más a la derecha)
(OR $s_1$ $s_2$ $s_3$ ... $s_n$)
En el caso de evaluar a verdadero, devuelve el primer valor distinto de NIL obtenido (el de más a la izquierda)

**Estructuras condicionales**

(IF Condicion ResultadoSiT ResultadoSiNoT)

```
(COND   (Condicion1 Resultado1)
        (Condicion2 Resultado2)
        …
        (Condicionn Resultadon))
```

Si ninguna condición evalúa a verdadero devuelve NIL
Para hacer condiciones exhaustivas, la $Condición_n$ puede ser T

## Definición de funciones

(DEFUN NombreFuncion($s_1$ $s_2$ $s_3$ … $s_n$) Descripción)

(lambda (x) (* 2 x))

**Llamadas a funciones**

(funcall *función* $s_1$ $s_2$ $s_3$ ... $s_n$)
(apply *función* $s_1$ $s_2$ $s_3$ ... $s_n$)
(*NombreFuncion* $s_1$ $s_2$ $s_3$ ... $s_n$)

Obtención de función a partir del nombre:

#'nombre o FUNCTION nombre